

Objetos literales

GUÍA DE EJEMPLOS EXPLICADOS · PASO A PASO

Doce ejemplos para entender los objetos.

Cada ejemplo explica primero la teoría y lo que tienes que hacer, y después te guía paso a paso por el código, línea a línea.

01 Lee el objetivo del ejemplo

02 Escribe el código paso a paso

03 Comprueba el resultado y prueba



JAVASCRIPT · OBJETOS LITERALES · LARAGON · NAVEGADOR (F12)

Cada ejemplo tiene su carpeta con index.html, style.css y app.js.
Complementa al Manual de objetos literales y a las Prácticas Guiadas.

Alpe formación



Tu ruta de trabajo

ÍNDICE Y REQUISITOS

Esta guía te acompaña en los **12 ejemplos** de objetos literales. En cada uno verás el **objetivo teórico**, el **objetivo práctico**, los pasos para realizarlo y la explicación del código que escribes.



Ejemplos

01 · Creación y anatomía 5–9

02 · Tipos de valores y claves 10–14

03 · Lectura y acceso 15–18

04 · Modificar, añadir y borrar 19–23

05 · Métodos y la palabra this 24–28

06 · Objetos y arrays dentro de objetos 29–32

07 · Arrays de objetos 33–36

08 · Recorrer un objeto 37–40

09 · Referencias y copias 41–45

10 · Desestructuración y spread 46–51

11 · Getters, setters y privacidad 52–59

12 · JSON y localStorage 60–66

Cómo usar la guía: pág. 3 · Resumen de conceptos: pág. 67 · Fuentes: pág. 68



Antes de empezar

- [ ] Conoces las variables (`let`, `const`), las funciones y los arrays de JavaScript.
- [ ] Sabes abrir la consola del navegador con **F12**.
- [ ] Tienes **Laragon** y **Visual Studio Code** instalados.



Cómo usar esta guía

LA CARPETA DE EJEMPLOS Y LOS APARTADOS



Qué vas a encontrar en cada ejemplo

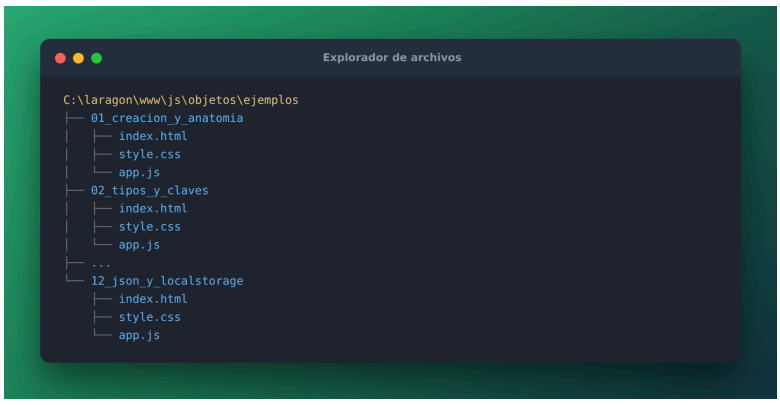
Apartado	Para qué sirve
Objetivo teórico	El concepto que se explica en el ejemplo
Objetivo práctico	Lo que tienes que hacer tú: lo que debe funcionar al terminar
Pasos	Cada paso es un trozo de código que escribes en `app.js`, con su explicación
Resultado esperado	Cómo debe verse la página y qué debe aparecer en el panel
Prueba tú	Pequeños cambios para comprobar que lo has entendido



Dónde se guardan los ejemplos

Cada ejemplo tiene su propia carpeta con tres archivos.

Copia la carpeta `ejemplos` en `C:\laragon\www\js\objetos\`.



Archivo	Estado	Tu trabajo
`index.html`	**Ya realizado**	No lo modifiques
`style.css`	**Ya realizado**	No lo modifiques
`app.js`	Vacío al empezar	**Es lo que escribes, paso a paso**



Cómo usar esta guía · La utilidad común

EL PANEL DE CONSOLA Y EL FLUJO DE TRABAJO



El panel de consola de cada página

Cada ejemplo muestra los resultados en dos sitios a la vez: la consola del navegador y un panel dentro de la página.

Así puedes ver los resultados sin abrir las herramientas de desarrollo. Aun así, abre la consola con `F12` para ver también los errores.

app.js · El panel de consola

```
// Reflejo en consola visual
const salida = document.getElementById("salidaConsola");
function logSalida(...args) {
  console.log(...args);
  salida.textContent += args.map(a => typeof a === "object" ? JSON.stringify(a, null, 2) : String
(a)).join(" ") + "\n";
}
```

- `salida` apunta al elemento `<pre id="salidaConsola">` del HTML.
- `logSalida()` recibe cualquier número de datos (`...args`) y los escribe en la consola y en el panel.
- Los objetos se convierten en texto con `JSON.stringify(a, null, 2)`; el resto de valores, con `String(a)`.



Cómo trabajar con cada ejemplo

1. **Preparar**
   - Crea la carpeta del ejemplo y copia en ella `index.html` y `style.css`.
   - Crea el archivo `app.js` vacío y ábrelo con Visual Studio Code.
2. **Escribir**
   - Escribe el código de cada paso en `app.js`, en el orden de la guía.
   - Guarda con `Ctrl + S` y recarga con `F5` tras cada paso.
3. **Comprobar**
   - Abre `http://localhost/js/objetos/ejemplos/<carpeta>/index.html`.
   - Compara con el resultado esperado y prueba los cambios de «Prueba tú».



Ejemplo 01 · Creación y anatomía

TEMAS 01 Y 02 · FUNDAMENTOS

OBJETIVO TEÓRICO

- Un objeto literal agrupa datos y acciones bajo un mismo nombre, escritos entre llaves `{ }`.
- Cada dato es una **propiedad** (`clave: valor`) y cada acción es un **método** (una función). Ambos son **miembros** del objeto.
- Hay tres formas habituales de crearlo: directa, vacío con asignación posterior y abreviada con variables (ES6). [1]

OBJETIVO PRÁCTICO · LO QUE TIENES QUE HACER

- Crear el objeto `coche` con dos propiedades y un método.
- Crear tres objetos más, uno con cada forma de creación.
- Mostrar los resultados en el panel de la página y en la consola.
- Conectar el botón `coche.arrancar()` con la tarjeta.



Paso 1 · Preparar la carpeta

1 **Crear la carpeta y los archivos**

- Crea la carpeta `01_creacion_y_anatomia` dentro de `C:\laragon\www\js\objetos\ejemplos\`.
- Copia en ella `index.html` y `style.css`, que ya están realizados.
- Crea el archivo `app.js` vacío y ábrelo con Visual Studio Code.
- En cada paso siguiente, escribe en `app.js` el código que se muestra.



Paso 2 · El panel de consola

app.js · El panel de consola

```
// Reflejo en consola visual
const salida = document.getElementById("salidaConsola");
function logSalida(...args) {
  console.log(...args);
  salida.textContent += args.map(a => typeof a === "object" ? JSON.stringify(a, null, 2) : String
(a)).join(" ") + "\n";
}
```

QUÉ HACE EL CÓDIGO

- `salida` guarda el elemento `<pre>` de la página donde se escribirá el texto: es el «panel de consola».
- `logSalida()` escribe cada mensaje en la consola real (`console.log`) y también en ese panel.
- Si el mensaje es un objeto, lo convierte en texto con `JSON.stringify` para poder verlo en pantalla.

Ejemplo 01 · Creación y anatomía (continuación)

PARTE 2 DE 5



Paso 3 · El objeto coche

app.js · El objeto coche

```
// 1. ANATOMÍA BÁSICA (Página 4 del manual)
const coche = {
  marca: "Toyota",
  puertas: 5,
  arrancar() {
    return "En marcha";
  }
};

logSalida("=== 1. ANATOMÍA DEL OBJETO COCHE ===");
logSalida("Marca:", coche.marca);
logSalida("Puertas:", coche.puertas);
logSalida("Acción arrancar():", coche.arrancar());
```

QUÉ HACE EL CÓDIGO

- `const coche = { ... }` crea el objeto con llaves.
- `marca` y `puertas` son propiedades: clave, dos puntos y valor.
- `arrancar() { ... }` es un método: una función dentro del objeto, sin la palabra `function`.
- Las propiedades se leen con punto (`coche.marca`). El método se llama con paréntesis: `coche.arrancar()`.

Ejemplo 01 · Creación y anatomía (continuación)

PARTE 3 DE 5



Paso 4 · Las tres formas de crear un objeto

app.js · Las tres formas de crear un objeto

```
// 2. TRES FORMAS DE CREAR UN OBJETO LITERAL (Página 5 del manual)
logSalida("\n=== 2. LAS 3 FORMAS DE CREAR OBJETOS ===");

// Forma 1: Directa (la más habitual cuando se conocen los datos)
const usuarioDirecto = {
  nombre: "Ana",
  rol: "Alumna"
};
logSalida("Forma 1 (Directa):", usuarioDirecto);

// Forma 2: Vacío y asignación posterior (útil con formularios)
const usuarioVacio = {};
usuarioVacio.nombre = "Marcos";
usuarioVacio.rol = "Profesor";
logSalida("Forma 2 (Vacío y asignación):", usuarioVacio);

// Forma 3: Abreviada ES6 (cuando la variable se llama igual que la clave)
const marca = "Logitech";
const modelo = "MX Master 3";
const raton = { marca, modelo }; // Equivale a { marca: marca, modelo: modelo }
logSalida("Forma 3 (Abreviada ES6):", raton);
```

QUÉ HACE EL CÓDIGO

- **Forma 1, directa:** se escriben las propiedades al crear el objeto. Es la más habitual si conoces los datos.
- **Forma 2, vacío y asignación:** se crea `{}` y después se añaden propiedades con `objeto.clave = valor`. Útil con formularios.
- **Forma 3, abreviada:** si la variable se llama igual que la clave, basta con escribir `{ marca, modelo }`.

Ejemplo 01 · Creación y anatomía (continuación)

PARTE 4 DE 5



Paso 5 · Conectar el objeto con la página

app.js · Conectar el objeto con la página

```
// Interacción en la interfaz
document.getElementById("txtMarca").textContent = coche.marca;
document.getElementById("txtPuertas").textContent = coche.puertas;
document.getElementById("txtEstado").textContent = "Parado";

document.getElementById("btnArrancar").addEventListener("click", () => {
  const estado = coche.arrancar();
  document.getElementById("txtEstado").textContent = estado;
  logSalida(`\n🚗 ¡Botón pulsado! coche.arrancar() -> "${estado}"`);
});
```

QUÉ HACE EL CÓDIGO

- Las tres primeras líneas copian los valores del objeto en los `<span>` de la tarjeta con `textContent`.
- El escuchador de `click` llama a `coche.arrancar()` y muestra lo que devuelve en el estado y en el panel.
- Fíjate: el método solo devuelve un texto. La página decide dónde mostrarlo.



Paso 6 · Guardar, abrir y comprobar

6

Ejecutar el ejemplo

- Guarda `app.js` con `Ctrl + S`.
- Comprueba en Laragon que Apache está encendido.
- Abre `http://localhost/js/objetos/ejemplos/01_creacion_y_anatomia/index.html` y pulsa los botones.
- Abre la consola con `F12` y compara con el panel.

Ejemplo 01 · Creación y anatomía (continuación)

PARTE 5 DE 5



Comprueba y practica

RESULTADO ESPERADO



QUÉ DEBES VER

- La tarjeta muestra Toyota, 5 puertas y el estado «Parado».
- El panel muestra las secciones 1 y 2 con los objetos creados.
- Al pulsar el botón, el estado cambia a «En marcha» y se añade una línea al panel.

PRUEBA TÚ

- Añade a `coche` la propiedad `color` y muéstrala en la consola.
- Crea un cuarto objeto con la forma abreviada usando dos variables tuyas.
- Añade el método `parar()` que devuelva "Parado".



Ejemplo 02 · Tipos de valores y nombres de clave

TEMA 02 · VALORES Y CLAVES

OBJETIVO TEÓRICO

- El valor de una propiedad puede ser de cualquier tipo: texto, número, booleano, `null`, `undefined`, un array, otro objeto o una función.
- La clave no necesita comillas si es un identificador válido. Si lleva espacios o guiones, va entre comillas y solo se lee con corchetes.
- Una clave **calculada** se escribe entre corchetes y su nombre sale de una expresión: `[prefijo + "_" + id]`.

OBJETIVO PRÁCTICO · LO QUE TIENES QUE HACER

- Crear un objeto con valores de todos los tipos y mostrar algunos.
- Crear un objeto con claves normales, con espacios y con guiones, y leerlas con la notación adecuada.
- Crear una clave calculada a partir de variables.
- Probar el generador de claves de la página con distintos prefijos.



Paso 1 · Preparar la carpeta

1

Crear la carpeta y los archivos

- Crea la carpeta `02_tipos_y_claves` dentro de `C:\laragon\www\js\objetos\ejemplos\`.
- Copia en ella `index.html` y `style.css`, que ya están realizados.
- Crea el archivo `app.js` vacío y ábrelo con Visual Studio Code.
- En cada paso siguiente, escribe en `app.js` el código que se muestra.



Paso 2 · El panel de consola

app.js · El panel de consola

```
const salida = document.getElementById("salidaConsola");
function logSalida(...args) {
  console.log(...args);
  salida.textContent += args.map(a => typeof a === "object" ? JSON.stringify(a, null, 2) : String
(a)).join(" ") + "\n";
}
```

QUÉ HACE EL CÓDIGO

- Es la misma utilidad del ejemplo 01: escribe en la consola y en el panel de la página.

Ejemplo 02 · Tipos de valores y nombres de clave (continuación)

PARTE 2 DE 5



Paso 3 · Tipos de valores

app.js · Tipos de valores

```
// 1. DIVERSIDAD DE TIPOS DE VALORES (Página 6 del manual)
const entidadCompleta = {
  texto: "Hola mundo",
  numero: 42,
  booleano: true,
  nulo: null,
  indefinido: undefined,
  array: [1, 2, 3],
  objetoInterno: { subclave: "valor interno" },
  metodo() { return "Soy una función dentro de un objeto"; }
};

logSalida("=== 1. TIPOS DE VALORES ADMITIDOS ===");
logSalida("Texto:", entidadCompleta.texto);
logSalida("Número:", entidadCompleta.numero);
logSalida("Array:", entidadCompleta.array);
logSalida("Objeto interno:", entidadCompleta.objetoInterno.subclave);
```

QUÉ HACE EL CÓDIGO

- `entidadCompleta` reúne un valor de cada tipo.
- `nulo: null` es una ausencia de valor intencionada; `indefinido: undefined` es un valor sin definir.
- `objetoInterno` es un objeto dentro de otro: se llega con `entidadCompleta.objetoInterno.subclave`.
- `metodo()` es una función guardada en el objeto: por eso es un método.

Ejemplo 02 · Tipos de valores y nombres de clave (continuación)

PARTE 3 DE 5



Paso 4 · Nombres de clave y comillas

app.js · Nombres de clave y comillas

```
// 2. NOMBRES DE CLAVE Y REGLAS DE COMILLAS (Página 7 del manual)
const clavesEspeciales = {
  identificadorValido: 100,
  "clave con espacios": "Necesita comillas al definirse",
  "clave-con-guiones": "Debe leerse obligatoriamente con corchetes"
};

logSalida("\n=== 2. NOMBRES DE CLAVE Y COMILLAS ===");
logSalida("Clave normal (punto):", clavesEspeciales.identificadorValido);
logSalida("Clave con espacios (corchetes):", clavesEspeciales["clave con espacios"]);
logSalida("Clave con guiones (corchetes):", clavesEspeciales["clave-con-guiones"]);
```

QUÉ HACE EL CÓDIGO

- `identificadorValido` no lleva comillas.
- `"clave con espacios"` y `"clave-con-guiones"` necesitan comillas porque no son identificadores válidos.
- Esas claves solo se pueden leer con corchetes: `clavesEspeciales["clave con espacios"]`.



Paso 5 · Clave calculada

app.js · Clave calculada

```
// 3. CLAVES CALCULADAS (ES6)
const idDinamico = 42;
const prefijo = "campo";

const objetoCalculado = {
  [prefijo + "_" + idDinamico]: "Valor generado dinámicamente"
};

logSalida("\n=== 3. CLAVE CALCULADA CON CORCHETES ===");
logSalida(objetoCalculado);
```

QUÉ HACE EL CÓDIGO

- Los corchetes en la definición indican que el nombre de la clave se calcula: `[prefijo + "_" + idDinamico]`.
- El resultado es una clave llamada `campo_42`.

Ejemplo 02 · Tipos de valores y nombres de clave (continuación)

PARTE 4 DE 5



Paso 6 · El generador interactivo

app.js · El generador interactivo

```
// Interacción
const inpPrefijo = document.getElementById("inpPrefijo");
const btnGenerar = document.getElementById("btnGenerar");
const resultadoClave = document.getElementById("resultadoClave");

btnGenerar.addEventListener("click", () => {
  const p = inpPrefijo.value.trim() || "clave";
  const timestamp = Date.now();
  const obj = {
    [`${p}_${timestamp}`]: "Dato generado a las " + new Date().toLocaleTimeString()
  };
  resultadoClave.innerHTML = `<code>${JSON.stringify(obj)}</code>`;
  logSalida("Nueva clave calculada generada:", obj);
});
```

QUÉ HACE EL CÓDIGO

- Lee el prefijo del `<input>` y, si está vacío, usa «clave».
- `Date.now()` da un número distinto en cada pulsación, así que la clave nunca se repite.
- La clave `[ ${p}_${timestamp} ]` se calcula con una plantilla de texto.
- Nota: el resultado se pinta con `innerHTML`. Si el texto lo escribe una persona, en una aplicación real conviene usar `textContent`.



Paso 7 · Guardar, abrir y comprobar

7

Ejecutar el ejemplo

- Guarda `app.js` con `Ctrl + S`.
- Comprueba en Laragon que Apache está encendido.
- Abre `http://localhost/js/objetos/ejemplos/02_tipos_y_claves/index.html` y pulsa los botones.
- Abre la consola con `F12` y compara con el panel.

Ejemplo 02 · Tipos de valores y nombres de clave (continuación)

PARTE 5 DE 5



Comprueba y practica

RESULTADO ESPERADO



QUÉ DEBES VER

- El panel muestra las secciones 1 a 3.
- Al pulsar «Generar Clave Calculada» aparece un objeto con una clave nueva, que empieza por el prefijo escrito.

PRUEBA TÚ

- Cambia el prefijo a `alumno` y genera dos claves.
- Añade a `clavesEspeciales` otra clave con un guion y léela con corchetes.
- Lee `clavesEspeciales.clave-con-guiones` con punto y observa el error en la consola.



Ejemplo 03 · Lectura y acceso a propiedades

TEMA 03 · PUNTO, CORCHETES Y ?.

OBJETIVO TEÓRICO

- Para leer una propiedad hay dos notaciones: punto (`obj.clave`) y corchetes (`obj[clave]`).
- Los corchetes permiten usar el nombre de la clave guardado en una variable.
- Si la propiedad no existe se obtiene `undefined` . Y con `?.` se evita el error al leer un dato de algo que no existe. [1]

OBJETIVO PRÁCTICO · LO QUE TIENES QUE HACER

- Crear el objeto `libro` y leer sus propiedades con punto.
- Leer una propiedad con corchetes usando una variable.
- Comprobar qué devuelve una propiedad que no existe.
- Usar `?.` para leer un dato que falta sin romper el programa.
- Consultar propiedades desde el selector de la página.



Paso 1 · Preparar la carpeta

1 Crear la carpeta y los archivos

- Crea la carpeta `03_lectura_y_acceso` dentro de `C:\laragon\www\js\objetos\ejemplos\` .
- Copia en ella `index.html` y `style.css` , que ya están realizados.
- Crea el archivo `app.js` vacío y ábrelo con Visual Studio Code.
- En cada paso siguiente, escribe en `app.js` el código que se muestra.



Paso 2 · El panel de consola

app.js · El panel de consola

```
const salida = document.getElementById("salidaConsola");
function logSalida(...args) {
  console.log(...args);
  salida.textContent += args.map(a => typeof a === "object" ? JSON.stringify(a, null, 2) : String
(a)).join(" ") + "\n";
}
```

QUÉ HACE EL CÓDIGO

- Misma utilidad que en los ejemplos anteriores.

Ejemplo 03 · Lectura y acceso a propiedades (continuación)

PARTE 2 DE 4



Paso 3 · Notación de punto

app.js · Notación de punto

```
const libro = {
  titulo: "Cien años de soledad",
  autor: "Gabriel García Márquez",
  año: 1967
};

// 1. NOTACIÓN DE PUNTO (la preferida cuando conocemos el nombre de la clave)
logSalida("=== 1. NOTACIÓN DE PUNTO ===");
logSalida("libro.titulo ->", libro.titulo);
logSalida("libro.autor  ->", libro.autor);
```

QUÉ HACE EL CÓDIGO

- `libro` es el objeto con título, autor y año.
- `libro.titulo` y `libro.autor` leen cada propiedad con punto: es la forma preferida si conoces el nombre.



Paso 4 · Corchetes y propiedades inexistentes

app.js · Corchetes y propiedades inexistentes

```
// 2. NOTACIÓN DE CORCHETES (cuando el nombre está en una variable)
logSalida("\n=== 2. NOTACIÓN DE CORCHETES CON VARIABLES ===");
const campo = "año";
logSalida(`libro["${campo}"] ->`, libro[campo]);

// ¿Qué ocurre si la propiedad no existe? Devuelve undefined (sin romper el código)
logSalida("libro.editorial (no existe) ->", libro.editorial);
```

QUÉ HACE EL CÓDIGO

- `const campo = "año"` guarda el nombre de la clave en una variable.
- `libro[campo]` usa el valor de la variable como clave. Con punto, `libro.campo` buscaría una clave llamada «campo».
- `libro.editorial` no existe: devuelve `undefined` y el programa sigue funcionando.

Ejemplo 03 · Lectura y acceso a propiedades (continuación)

PARTE 3 DE 4



Paso 5 · Encadenamiento opcional (?.)

app.js · Encadenamiento opcional (?.)

```
// 3. ENCADENAMIENTO OPCIONAL (?.)
logSalida("\n=== 3. ENCADENAMIENTO OPCIONAL (?.) ===");
const usuario = {
  nombre: "Carlos"
  // No tiene propiedad 'perfil'
};

// usuario.perfil.avatar lanzaría TypeError: Cannot read properties of undefined
// Con ?. devuelve undefined de forma segura sin detener el script
logSalida("usuario.perfil?.avatar ->", usuario.perfil?.avatar);
```

QUÉ HACE EL CÓDIGO

- `usuario` no tiene la propiedad `perfil`.
- `usuario.perfil.avatar` lanzaría un `TypeError`, porque `perfil` es `undefined`.
- `usuario.perfil?.avatar` se detiene en `perfil` y devuelve `undefined`, sin error.



Paso 6 · Consulta interactiva

app.js · Consulta interactiva

```
// Interacción
const selPropiedad = document.getElementById("selPropiedad");
const btnConsultar = document.getElementById("btnConsultar");
const cajaResultado = document.getElementById("cajaResultado");

btnConsultar.addEventListener("click", () => {
  const clave = selPropiedad.value;
  const valor = libro[clave];
  cajaResultado.innerHTML = `<code>libro["${clave}"]</code> = <strong>${valor}</strong> (tipo: ${type
of valor})`;
  logSalida(`Consulta interactiva: libro["${clave}"] -> ${valor}`);
});
```

QUÉ HACE EL CÓDIGO

- El `<select>` guarda el nombre de la propiedad elegida.
- `libro[clave]` lee la propiedad cuyo nombre está en la variable `clave`.
- `typeof valor` indica el tipo del resultado; para `editorial` será `undefined`.

Ejemplo 03 · Lectura y acceso a propiedades (continuación)

PARTE 4 DE 4



Paso 7 · Guardar, abrir y comprobar

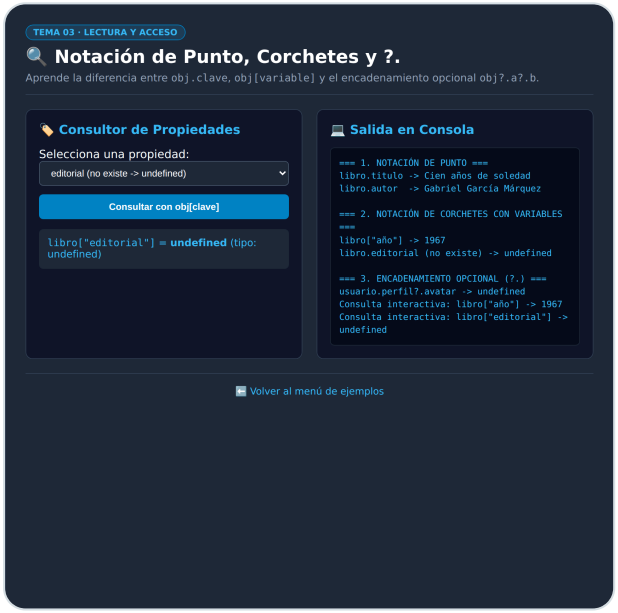
7 Ejecutar el ejemplo

- Guarda `app.js` con `Ctrl + S`.
- Comprueba en Laragon que Apache está encendido.
- Abre `http://localhost/js/objetos/ejemplos/03_lectura_y_acceso/index.html` y pulsa los botones.
- Abre la consola con `F12` y compara con el panel.



Comprueba y practica

RESULTADO ESPERADO



QUÉ DEBES VER

- El panel muestra las tres secciones.
- Al consultar «editorial», el resultado es `undefined` y no aparece ningún error.

PRUEBA TÚ

- Consulta «año» y observa el tipo del valor.
- Añade una propiedad `isbn` al libro y una opción nueva en el `<select>`.
- Quita el `?.` de `usuario.perfil?.avatar` y lee el error de la consola.



Ejemplo 04 · Modificar, añadir y borrar propiedades

TEMA 04 · OBJETOS DINÁMICOS

OBJETIVO TEÓRICO

- Un objeto es dinámico: puedes modificar valores, añadir propiedades nuevas y borrar las existentes.
- `delete obj.clave` elimina una propiedad y `"clave" in obj` comprueba si existe.
- `Object.freeze(obj)` congela el objeto: impide modificarlo, añadir propiedades y borrarlas.

OBJETIVO PRÁCTICO · LO QUE TIENES QUE HACER

- Crear el objeto `producto` y mostrarlo en pantalla.
- Modificar una propiedad, añadir otra y borrar una tercera.
- Comprobar con `in` que una propiedad borrada ya no existe.
- Congelar el objeto y comprobar que ya no se puede modificar.



Paso 1 · Preparar la carpeta

1 Crear la carpeta y los archivos

- Crea la carpeta `04_modificar_y_borrar` dentro de `C:\laragon\www\js\objetos\ejemplos\`.
- Copia en ella `index.html` y `style.css`, que ya están realizados.
- Crea el archivo `app.js` vacío y ábrelo con Visual Studio Code.
- En cada paso siguiente, escribe en `app.js` el código que se muestra.



Paso 2 · El panel de consola

app.js · El panel de consola

```
const salida = document.getElementById("salidaConsola");
const estadoProducto = document.getElementById("estadoProducto");

function logSalida(...args) {
  console.log(...args);
  salida.textContent += args.map(a => typeof a === "object" ? JSON.stringify(a, null, 2) : String
(a)).join(" ") + "\n";
}
```

QUÉ HACE EL CÓDIGO

- Es la utilidad de siempre, más una constante, `estadoProducto`, que apunta al elemento donde se verá el objeto.

Ejemplo 04 · Modificar, añadir y borrar propiedades (continuación)

PARTE 2 DE 5



Paso 3 · El objeto producto y su vista

app.js · El objeto producto y su vista

```
let producto = {
  codigo: "A-104",
  nombre: "Teclado Mecánico",
  precio: 59.90,
  enStock: true
};

function refrescarVista() {
  estadoProducto.textContent = JSON.stringify(producto, null, 2);
}

logSalida("=== ESTADO INICIAL DEL OBJETO ===");
logSalida(producto);
```

QUÉ HACE EL CÓDIGO

- `producto` tiene cuatro propiedades. Se declara con `let`; con `const` funcionaría igual, porque `const` impide reasignar la variable, no modificar su contenido.
- `refrescarVista()` pinta el objeto en la tarjeta como texto JSON con sangría.



Paso 4 · Modificar una propiedad

app.js · Modificar una propiedad

```
// 1. Modificar
document.getElementById("btnModificar").addEventListener("click", () => {
  producto.precio = 49.95;
  refrescarVista();
  logSalida("✔ producto.precio modificado a 49.95:", producto);
});
```

QUÉ HACE EL CÓDIGO

- `producto.precio = 49.95` cambia el valor de una propiedad que ya existe.
- Después se refresca la vista y se escribe el objeto en el panel.

Ejemplo 04 · Modificar, añadir y borrar propiedades (continuación)

PARTE 3 DE 5



Paso 5 · Añadir una propiedad

app.js · Añadir una propiedad

```
// 2. Añadir nueva propiedad
document.getElementById("btnAgregar").addEventListener("click", () => {
  producto.categoria = "Periféricos";
  refrescarVista();
  logSalida("✔ Nueva propiedad 'categoria' añadida:", producto);
});
```

QUÉ HACE EL CÓDIGO

- Asignar a una clave que no existe la crea: `producto.categoria = "Periféricos"`.



Paso 6 · Borrar y comprobar con in

app.js · Borrar y comprobar con in

```
// 3. Borrar propiedad con delete y comprobar con 'in'
document.getElementById("btnBorrar").addEventListener("click", () => {
  delete producto.codigo;
  refrescarVista();
  logSalida("🗑️ delete producto.codigo ejecutado.");
  logSalida("¿Sigue existiendo 'codigo' in producto? ->", "codigo" in producto);
});
```

QUÉ HACE EL CÓDIGO

- `delete producto.codigo` elimina la propiedad.
- `"codigo" in producto` devuelve `false` cuando la clave ya no existe.

Ejemplo 04 · Modificar, añadir y borrar propiedades (continuación)

PARTE 4 DE 5



Paso 7 · Congelar el objeto

app.js · Congelar el objeto

```
// 4. Object.freeze()
document.getElementById("btnCongelar").addEventListener("click", () => {
  Object.freeze(producto);
  logSalida("❄ Object.freeze(producto) aplicado. El objeto es ahora inmutable.");
  try {
    producto.precio = 999; // Falla silenciosamente o lanza TypeError en strict mode
    logSalida("Intento de cambiar precio tras freeze. Precio actual:", producto.precio);
  } catch (e) {
    logSalida("Error al modificar objeto congelado:", e.message);
  }
});

refrescarVista();
```

QUÉ HACE EL CÓDIGO

- `Object.freeze(producto)` convierte el objeto en inmutable.
- Después, `producto.precio = 999` se ignora (en modo estricto lanzaría un `TypeError`).
- El `try...catch` captura el error si lo hubiera. El panel muestra el precio, que no ha cambiado.



Paso 8 · Guardar, abrir y comprobar

8

Ejecutar el ejemplo

- Guarda `app.js` con `Ctrl + S`.
- Comprueba en Laragon que Apache está encendido.
- Abre `http://localhost/js/objetos/ejemplos/04_modificar_y_borrar/index.html` y pulsa los botones.
- Abre la consola con `F12` y compara con el panel.

Ejemplo 04 · Modificar, añadir y borrar propiedades (continuación)

PARTE 5 DE 5



Comprueba y practica

RESULTADO ESPERADO



QUÉ DEBES VER

- Cada botón cambia la tarjeta de estado y añade líneas al panel.
- Tras congelar, el precio no cambia aunque se intente modificar.

PRUEBA TÚ

- Pulsa los botones en otro orden: borra primero y modifica después.
- Añade la propiedad `descuento` y bórrala.
- Tras congelar, pulsa «Añadir Categoría» y comprueba que ya no tiene efecto.



Ejemplo 05 · Métodos y la palabra this

TEMA 05 · FUNCIONES DENTRO DE OBJETOS

OBJETIVO TEÓRICO

- Un método es una función guardada en una propiedad. Cuando se llama con `objeto.metodo()`, la palabra `this` apunta a ese objeto. [2]
- Por eso un método puede leer y modificar las propiedades del objeto con `this.vida`.
- Las funciones flecha no tienen su propio `this`: no son adecuadas como métodos. [3]

OBJETIVO PRÁCTICO · LO QUE TIENES QUE HACER

- Crear el objeto `personaje` con propiedades y dos métodos que usan `this`.
- Conectar los botones para dañar y curar al personaje y actualizar la vida en pantalla.
- Comprobar qué ocurre con un método escrito como función flecha.



Paso 1 · Preparar la carpeta

1 **Crear la carpeta y los archivos**

- Crea la carpeta `05_metodos_y_this` dentro de `C:\laragon\www\js\objetos\ejemplos\`.
- Copia en ella `index.html` y `style.css`, que ya están realizados.
- Crea el archivo `app.js` vacío y ábrelo con Visual Studio Code.
- En cada paso siguiente, escribe en `app.js` el código que se muestra.



Paso 2 · El panel de consola

app.js · El panel de consola

```
const salida = document.getElementById("salidaConsola");
const txtVida = document.getElementById("txtVida");

function logSalida(...args) {
  console.log(...args);
  salida.textContent += args.map(a => typeof a === "object" ? JSON.stringify(a, null, 2) : String
(a)).join(" ") + "\n";
}
```

QUÉ HACE EL CÓDIGO

- La utilidad de siempre, más `txtVida`, el elemento donde se verá la vida.

Ejemplo 05 · Métodos y la palabra this (continuación)

PARTE 2 DE 5



Paso 3 · El objeto personaje

app.js · El objeto personaje

```
const personaje = {
  nombre: "Valeria",
  vida: 100,

  // Método abreviado estándar (RECOMENDADO): 'this' apunta a 'personaje'
  recibirDanio(puntos) {
    this.vida -= puntos;
    if (this.vida < 0) this.vida = 0;
    return `${this.nombre} recibe ${puntos} de daño. Vida restante: ${this.vida}`;
  },

  curar(puntos) {
    this.vida += puntos;
    if (this.vida > 100) this.vida = 100;
    return `${this.nombre} se cura ${puntos} puntos. Vida actual: ${this.vida}`;
  },

  // ERROR TÍPICO: Función flecha como método
  // Las funciones flecha no tienen su propio 'this', heredan el global (window)
  metodoFlechaErroneo: () => {
    return `this.nombre en arrow function da: ${this ? this.nombre : 'undefined'}`;
  }
};
```

QUÉ HACE EL CÓDIGO

- `nombre` y `vida` son las propiedades.
- `recibirDanio(puntos)` resta con `this.vida -= puntos` y evita que la vida baje de 0.
- `curar(puntos)` suma y evita que pase de 100.
- Los dos devuelven un mensaje con `return`: lo mostrará quien llame al método.
- `metodoFlechaErroneo` está escrito con función flecha: es el **error típico** que vamos a provocar.

Ejemplo 05 · Métodos y la palabra this (continuación)

PARTE 3 DE 5



Paso 4 · Estado inicial y botón de daño

app.js · Estado inicial y botón de daño

```
logSalida("=== ESTADO INICIAL DEL PERSONAJE ===");
logSalida(`Héroe: ${personaje.nombre}, Salud: ${personaje.vida}`);

document.getElementById("btnDanio").addEventListener("click", () => {
  const msg = personaje.recibirDanio(20);
  txtVida.textContent = personaje.vida;
  logSalida("⚔️", msg);
});
```

QUÉ HACE EL CÓDIGO

- Primero se escribe el estado inicial en el panel.
- El botón llama a `personaje.recibirDanio(20)` y guarda el mensaje.
- Actualiza la vida de la tarjeta con `txtVida.textContent` y escribe el mensaje en el panel.



Paso 5 · Botón de curar

app.js · Botón de curar

```
document.getElementById("btnCurar").addEventListener("click", () => {
  const msg = personaje.curar(15);
  txtVida.textContent = personaje.vida;
  logSalida("🧪", msg);
});
```

QUÉ HACE EL CÓDIGO

- Funciona igual, con `personaje.curar(15)`.
- Los dos métodos cambian `this.vida` del mismo objeto.

Ejemplo 05 · Métodos y la palabra this (continuación)

PARTE 4 DE 5



Paso 6 · El error de la función flecha

app.js · El error de la función flecha

```
document.getElementById("btnFlecha").addEventListener("click", () => {
  logSalida("❌ Ejecutando personaje.metodoFlechaErroneo():");
  logSalida(personaje.metodoFlechaErroneo());
  logSalida("⚠️ Explicación: En una función flecha, 'this' apunta a 'window' y no al objeto 'personaj
e'.");
});
```

QUÉ HACE EL CÓDIGO

- La función flecha no tiene `this` propio: usa el del contexto exterior, que en una página es la ventana (`window`).
- Por eso `this.nombre` no devuelve «Valeria», sino `undefined` .
- Solución: escribir el método con la forma abreviada, como `recibirDanio(puntos) { ... }` . [2] [3]



Paso 7 · Guardar, abrir y comprobar

7 **Ejecutar el ejemplo**

- Guarda `app.js` con `Ctrl + S` .
- Comprueba en Laragon que Apache está encendido.
- Abre `http://localhost/js/objetos/ejemplos/05_metodos_y_this/index.html` y pulsa los botones.
- Abre la consola con `F12` y compara con el panel.

Ejemplo 05 · Métodos y la palabra this (continuación)

PARTE 5 DE 5



Comprueba y practica

RESULTADO ESPERADO



QUÉ DEBES VER

- La vida baja de 20 en 20 y sube de 15 en 15, sin salirse de 0 ni de 100.
- El botón del error muestra `undefined` en lugar del nombre.

PRUEBA TÚ

- Pulsa seis veces «Recibir Daño» y comprueba que la vida se queda en 0.
- Reescribe `metodoFlechaErroneo` como método abreviado y comprueba que ya devuelve el nombre.
- Añade un método `estado()` que devuelva el nombre y la vida.



Ejemplo 06 · Objetos y arrays dentro de objetos

TEMA 06 · ESTRUCTURAS MULTINIVEL

OBJETIVO TEÓRICO

- Un objeto puede contener otros objetos y arrays. Se llega a un dato interno encadenando puntos y corchetes.
- `empresa.direccion.ciudad` entra en el objeto `direccion`; `empresa.empleados[0].nombre` entra en el primer elemento del array.
- Si un nivel intermedio no existe, `?.` evita el `TypeError`.

OBJETIVO PRÁCTICO · LO QUE TIENES QUE HACER

- Crear el objeto `empresa` con un objeto anidado, un array de objetos y un método.
- Leer datos de distintos niveles.
- Añadir un empleado al array y repintar la ficha.



Paso 1 · Preparar la carpeta

1 Crear la carpeta y los archivos

- Crea la carpeta `06_objetos_anidados` dentro de `C:\laragon\www\js\objetos\ejemplos\`.
- Copia en ella `index.html` y `style.css`, que ya están realizados.
- Crea el archivo `app.js` vacío y ábrelo con Visual Studio Code.
- En cada paso siguiente, escribe en `app.js` el código que se muestra.



Paso 2 · El panel de consola

app.js · El panel de consola

```
const salida = document.getElementById("salidaConsola");
const fichaEmpresa = document.getElementById("fichaEmpresa");

function logSalida(...args) {
  console.log(...args);
  salida.textContent += args.map(a => typeof a === "object" ? JSON.stringify(a, null, 2) : String
(a)).join(" ") + "\n";
}
```

QUÉ HACE EL CÓDIGO

- La utilidad de siempre, más `fichaEmpresa`, el elemento donde se pintará la ficha.

Ejemplo 06 · Objetos y arrays dentro de objetos (continuación)

PARTE 2 DE 4



Paso 3 · La estructura de la empresa

app.js · La estructura de la empresa

```
const empresa = {
  nombre: "Alpe Formación",
  direccion: {
    calle: "Paseo de Pereda 12",
    ciudad: "Santander",
    cp: "39004"
  },
  empleados: [
    { id: 1, nombre: "Lucía", puesto: "Docente" },
    { id: 2, nombre: "Marcos", puesto: "Coordinador" }
  ],
  obtenerPlantilla() {
    return this.empleados.map(e => `${e.nombre} (${e.puesto})`).join(", ");
  }
};
```

QUÉ HACE EL CÓDIGO

- `direccion` es un objeto dentro de `empresa`.
- `empleados` es un array y cada elemento es un objeto con `id`, `nombre` y `puesto`.
- `obtenerPlantilla()` usa `this.empleados` y `map` para devolver un texto con todos los nombres.



Paso 4 · Pintar la ficha

app.js · Pintar la ficha

```
function actualizarVista() {
  fichaEmpresa.innerHTML = `
    <p><strong>Empresa:</strong> ${empresa.nombre}</p>
    <p><strong>Sede:</strong> ${empresa.direccion.calle}, ${empresa.direccion.ciudad} (${empresa.direccion.cp})</p>
    <p><strong>Plantilla (${empresa.empleados.length} miembros):</strong> ${empresa.obtenerPlantilla()}</p>
  `;
}
```

QUÉ HACE EL CÓDIGO

- `actualizarVista()` construye el HTML de la ficha leyendo datos de varios niveles.
- Se vuelve a llamar cada vez que cambia el array, para que la pantalla refleje el estado actual.

Ejemplo 06 · Objetos y arrays dentro de objetos (continuación)

PARTE 3 DE 4



Paso 5 · Acceso a datos anidados

app.js · Acceso a datos anidados

```
logSalida("=== ACCESO A ESTRUCTURAS ANIDADAS ===");
logSalida("Ciudad de la sede (puntos encadenados):", empresa.direccion.ciudad);
logSalida("Primer empleado (array + objeto):", empresa.empleados[0].nombre);
logSalida("Total plantilla:", empresa.obtenerPlantilla());

// Encadenamiento opcional con niveles inexistentes
logSalida("\nAcceso seguro con ?.:", empresa.sucursales?.madrid?.telefono); // undefined sin error
```

QUÉ HACE EL CÓDIGO

- `empresa.direccion.ciudad`: dos puntos encadenados.
- `empresa.empleados[0].nombre`: array y objeto.
- `empresa.sucursales?.madrid?.telefono`: `sucursales` no existe, así que el resultado es `undefined` sin error.



Paso 6 · Añadir un empleado

app.js · Añadir un empleado

```
document.getElementById("btnAñadirEmpleado").addEventListener("click", () => {
  const nuevoId = empresa.empleados.length + 1;
  const nuevo = { id: nuevoId, nombre: `Empleado #${nuevoId}`, puesto: "Técnico Web" };
  empresa.empleados.push(nuevo);
  actualizarVista();
  logSalida("➕ Nuevo empleado añadido al array interno:", nuevo);
});

actualizarVista();
```

QUÉ HACE EL CÓDIGO

- El nuevo `id` se calcula con la longitud del array más uno.
- `push` añade el objeto al array interno y `actualizarVista()` repinta la ficha.



Paso 7 · Guardar, abrir y comprobar

7

Ejecutar el ejemplo

- Guarda `app.js` con `Ctrl + S`.
- Comprueba en Laragon que Apache está encendido.
- Abre `http://localhost/js/objetos/ejemplos/06_objetos_anidados/index.html` y pulsa los botones.
- Abre la consola con `F12` y compara con el panel.

Ejemplo 06 · Objetos y arrays dentro de objetos (continuación)

PARTE 4 DE 4



Comprueba y practica

RESULTADO ESPERADO



QUÉ DEBES VER

- La ficha muestra empresa, sede y dos empleados.
- Cada pulsación añade un empleado y actualiza el número de la plantilla.

PRUEBA TÚ

- Cambia la ciudad desde la consola: `empresa.direccion.ciudad = "Torrelavega"`.
- Añade la propiedad `telefono` a `direccion` y muéstrala en la ficha.
- Añade un método `buscarEmpleado(id)` que devuelva el empleado con ese id.



Ejemplo 07 · Arrays de objetos

TEMA 07 · COLECCIONES DE DATOS

OBJETIVO TEÓRICO

- La estructura más habitual es un array donde cada elemento es un objeto con las mismas claves.
- Se procesa con métodos de array: `filter` (los que cumplen una condición), `map` (transforma cada elemento), `find` (el primero que cumple) y `reduce` (acumula un valor).
- Ninguno modifica el array original: devuelven un resultado nuevo.

OBJETIVO PRÁCTICO · LO QUE TIENES QUE HACER

- Crear el array `catalogo` con cuatro productos (objetos).
- Mostrar todos los productos con `map` y `join`.
- Obtener solo los disponibles con `filter`.
- Extraer los títulos con `map`.
- Calcular el importe total con `reduce`.



Paso 1 · Preparar la carpeta

1 Crear la carpeta y los archivos

- Crea la carpeta `07_arrays_de_objetos` dentro de `C:\laragon\www\js\objetos\ejemplos\`.
- Copia en ella `index.html` y `style.css`, que ya están realizados.
- Crea el archivo `app.js` vacío y ábrelo con Visual Studio Code.
- En cada paso siguiente, escribe en `app.js` el código que se muestra.



Paso 2 · El panel de consola

app.js · El panel de consola

```
const salida = document.getElementById("salidaConsola");
const cajaResultado = document.getElementById("cajaResultado");

function logSalida(...args) {
  console.log(...args);
  salida.textContent += args.map(a => typeof a === "object" ? JSON.stringify(a, null, 2) : String
(a)).join(" ") + "\n";
}
```

QUÉ HACE EL CÓDIGO

- La utilidad de siempre, más `cajaResultado`, la caja donde se verá cada resultado.

Ejemplo 07 · Arrays de objetos (continuación)

PARTE 2 DE 4



Paso 3 · El catálogo

app.js · El catálogo

```
const catalogo = [
  { id: 1, titulo: "Monitor 24 pulgadas", precio: 140, disponible: true },
  { id: 2, titulo: "Ratón Inalámbrico",   precio: 25,  disponible: false },
  { id: 3, titulo: "Alfombrilla XXL",     precio: 18,  disponible: true },
  { id: 4, titulo: "Teclado Mecánico",    precio: 60,  disponible: true }
];

logSalida("=== CATÁLOGO DE PRODUCTOS (4 ÍTEMS) ===");
logSalida(catalogo);
```

QUÉ HACE EL CÓDIGO

- Cada producto es un objeto con `id`, `titulo`, `precio` y `disponible`.
- Todos tienen las mismas claves: así se pueden procesar de la misma manera.



Paso 4 · Mostrar todos con map

app.js · Mostrar todos con map

```
// 1. Mostrar Todos
document.getElementById("btnTodos").addEventListener("click", () => {
  cajaResultado.innerHTML = catalogo.map(p => `• ${p.titulo} - ${p.precio}€ (${p.disponible ? '✔ Stoc
k' : '✖ Agotado'})`).join("<br>");
  logSalida("Listado completo:", catalogo);
});
```

QUÉ HACE EL CÓDIGO

- `map` transforma cada producto en un texto.
- El operador ternario `p.disponible ? ... : ...` elige el texto según el valor booleano.
- `join("<br>")` une los textos en uno solo, separados por saltos de línea.

Ejemplo 07 · Arrays de objetos (continuación)

PARTE 3 DE 4



Paso 5 · Filtrar con filter

app.js · Filtrar con filter

```
// 2. filter()
document.getElementById("btnDisponibles").addEventListener("click", () => {
  const disponibles = catalogo.filter(p => p.disponible);
  cajaResultado.innerHTML = disponibles.map(p => `• <strong>${p.titulo}</strong> - ${p.precio}€`).joi
n("<br>");
  logSalida("Artículos disponibles (filter):", disponibles);
});
```

QUÉ HACE EL CÓDIGO

- `catalogo.filter(p => p.disponible)` devuelve un array nuevo solo con los productos cuya condición es `true`.



Paso 6 · Extraer con map

app.js · Extraer con map

```
// 3. map()
document.getElementById("btnNombres").addEventListener("click", () => {
  const titulos = catalogo.map(p => p.titulo);
  cajaResultado.innerHTML = `Titulos extraídos: <code>${titulos.join(" | ")}</code>`;
  logSalida("Array de títulos (map):", titulos);
});
```

QUÉ HACE EL CÓDIGO

- `catalogo.map(p => p.titulo)` devuelve un array con un título por cada producto.



Paso 7 · Acumular con reduce

app.js · Acumular con reduce

```
// 4. reduce()
document.getElementById("btnTotal").addEventListener("click", () => {
  const total = catalogo.reduce((acc, p) => acc + p.precio, 0);
  cajaResultado.innerHTML = `Importe acumulado del catálogo: <strong>${total.toFixed(2)} €</strong>`;
  logSalida("Importe total (reduce):", `${total.toFixed(2)} €`);
});
```

QUÉ HACE EL CÓDIGO

- `reduce((acc, p) => acc + p.precio, 0)` recorre el array acumulando en `acc`; el segundo argumento, `0`, es el valor inicial.
- `toFixed(2)` muestra el total con dos decimales.

Ejemplo 07 · Arrays de objetos (continuación)

PARTE 4 DE 4



Paso 8 · Guardar, abrir y comprobar

8 Ejecutar el ejemplo

- Guarda `app.js` con `Ctrl + S`.
- Comprueba en Laragon que Apache está encendido.
- Abre `http://localhost/js/objetos/ejemplos/07_arrays_de_objetos/index.html` y pulsa los botones.
- Abre la consola con `F12` y compara con el panel.



Comprueba y practica

RESULTADO ESPERADO



QUÉ DEBES VER

- «Solo Disponibles» muestra tres productos.
- El importe acumulado del catálogo es 243.00 €.

PRUEBA TÚ

- Obtén con `filter` los productos de menos de 50 €.
- Busca con `find` el producto con `id` 3.
- Calcula con `reduce` el total solo de los productos disponibles.



Ejemplo 08 · Recorrer un objeto

TEMA 08 · KEYS, VALUES, ENTRIES Y FOR...IN

OBJETIVO TEÓRICO

- Un objeto no se recorre directamente con `for...of`. Para recorrerlo se obtienen sus claves, sus valores o sus entradas.
- `Object.keys(obj)` devuelve las claves, `Object.values(obj)` los valores y `Object.entries(obj)` parejas `[clave, valor]`.
- `for (const k in obj)` recorre las claves una a una.

OBJETIVO PRÁCTICO · LO QUE TIENES QUE HACER

- Crear el objeto `curso`.
- Obtener sus claves, valores y entradas y mostrarlos.
- Recorrer el objeto con `for...in` y leer cada valor con `curso[k]`.



Paso 1 · Preparar la carpeta

1

Crear la carpeta y los archivos

- Crea la carpeta `08_recorrer_objetos` dentro de `C:\laragon\www\js\objetos\ejemplos\`.
- Copia en ella `index.html` y `style.css`, que ya están realizados.
- Crea el archivo `app.js` vacío y ábrelo con Visual Studio Code.
- En cada paso siguiente, escribe en `app.js` el código que se muestra.



Paso 2 · El panel de consola

app.js · El panel de consola

```
const salida = document.getElementById("salidaConsola");
const cajaResultado = document.getElementById("cajaResultado");

function logSalida(...args) {
  console.log(...args);
  salida.textContent += args.map(a => typeof a === "object" ? JSON.stringify(a, null, 2) : String
(a)).join(" ") + "\n";
}
```

QUÉ HACE EL CÓDIGO

- La utilidad de siempre, más `cajaResultado`.

Ejemplo 08 · Recorrer un objeto (continuación)

PARTE 2 DE 4



Paso 3 · El objeto curso

app.js · El objeto curso

```
const curso = {
  codigo: "IFCD0110",
  titulo: "Confección y Publicación Web",
  horas: 560,
  modalidad: "Presencial"
};

logSalida("=== OBJETO CURSO ===");
logSalida(curso);
```

QUÉ HACE EL CÓDIGO

- `curso` tiene cuatro propiedades con valores de texto y de número.



Paso 4 · Object.keys

app.js · Object.keys

```
// 1. Object.keys()
document.getElementById("btnKeys").addEventListener("click", () => {
  const claves = Object.keys(curso);
  cajaResultado.innerHTML = `Claves: <code>${JSON.stringify(claves)}</code>`;
  logSalida("Object.keys(curso):", claves);
});
```

QUÉ HACE EL CÓDIGO

- `Object.keys(curso)` devuelve un array con los nombres de las propiedades.
- `JSON.stringify(claves)` permite mostrar ese array como texto.



Paso 5 · Object.values

app.js · Object.values

```
// 2. Object.values()
document.getElementById("btnValues").addEventListener("click", () => {
  const valores = Object.values(curso);
  cajaResultado.innerHTML = `Valores: <code>${JSON.stringify(valores)}</code>`;
  logSalida("Object.values(curso):", valores);
});
```

QUÉ HACE EL CÓDIGO

- `Object.values(curso)` devuelve un array con los valores, en el mismo orden que las claves.

Ejemplo 08 · Recorrer un objeto (continuación)

PARTE 3 DE 4



Paso 6 · Object.entries

app.js · Object.entries

```
// 3. Object.entries()
document.getElementById("btnEntries").addEventListener("click", () => {
  const entradas = Object.entries(curso);
  cajaResultado.innerHTML = entradas.map(([k, v]) => `• <strong>${k}:</strong> ${v}`).join("<br>");
  logSalida("Object.entries(curso):", entradas);
});
```

QUÉ HACE EL CÓDIGO

- `Object.entries(curso)` devuelve un array de parejas `[clave, valor]`.
- En el `map`, `([k, v])` separa cada pareja en dos variables.



Paso 7 · Bucle for...in

app.js · Bucle for...in

```
// 4. Bucle for...in
document.getElementById("btnForIn").addEventListener("click", () => {
  let lineas = [];
  logSalida("--- Bucle for (const clave in curso) ---");
  for (const k in curso) {
    lineas.push(`• curso[${k}] = ${curso[k]}`);
    logSalida(`${k}:`, curso[k]);
  }
  cajaResultado.innerHTML = lineas.join("<br>");
});
```

QUÉ HACE EL CÓDIGO

- `for (const k in curso)` asigna a `k` el nombre de cada propiedad.
- `curso[k]` usa corchetes porque `k` es una variable: con punto no funcionaría.
- Las líneas se acumulan en un array y se muestran juntas con `join("<br>")`.



Paso 8 · Guardar, abrir y comprobar

8 **Ejecutar el ejemplo**

- Guarda `app.js` con `Ctrl + S`.
- Comprueba en Laragon que Apache está encendido.
- Abre `http://localhost/js/objetos/ejemplos/08_recorrer_objetos/index.html` y pulsa los botones.
- Abre la consola con `F12` y compara con el panel.

Ejemplo 08 · Recorrer un objeto (continuación)

PARTE 4 DE 4



Comprueba y practica

RESULTADO ESPERADO



QUÉ DEBES VER

- Cada botón escribe su resultado en el panel y en la consola.
- Las claves, los valores y las entradas salen en el mismo orden.

PRUEBA TÚ

- Añade la propiedad `precio` al objeto y comprueba que aparece en los cuatro botones.
- Calcula con `Object.keys(curso).length` cuántas propiedades tiene.
- Muestra solo las propiedades cuyo valor sea un número.



Ejemplo 09 · Referencias y copias

TEMA 09 · ASIGNAR NO ES COPIAR

OBJETIVO TEÓRICO

- Una variable no guarda el objeto: guarda una referencia a él. Por eso `b = a` no copia nada: las dos variables apuntan al mismo objeto.
- La copia superficial `{ ...a }` duplica el primer nivel, pero los objetos anidados se siguen compartiendo.
- `structuredClone(a)` hace una copia profunda: duplica todos los niveles. [4]

OBJETIVO PRÁCTICO · LO QUE TIENES QUE HACER

- Crear una función que devuelva un objeto con un objeto anidado.
- Comprobar que modificar `copiaFalsa` modifica el original.
- Comprobar que la copia superficial protege el primer nivel, pero no el anidado.
- Comprobar que la copia profunda deja el original intacto.



Paso 1 · Preparar la carpeta

1 Crear la carpeta y los archivos

- Crea la carpeta `09_referencias_y_copias` dentro de `C:\laragon\www\js\objetos\ejemplos\`.
- Copia en ella `index.html` y `style.css`, que ya están realizados.
- Crea el archivo `app.js` vacío y ábrelo con Visual Studio Code.
- En cada paso siguiente, escribe en `app.js` el código que se muestra.

Ejemplo 09 · Referencias y copias (continuación)

PARTE 2 DE 5



Paso 2 · Utilidades del panel

app.js · Utilidades del panel

```
// Elementos del DOM
const btnReferencia = document.getElementById('btnReferencia');
const btnSpread = document.getElementById('btnSpread');
const btnClone = document.getElementById('btnClone');
const cajaResultado = document.getElementById('cajaResultado');
const salidaConsola = document.getElementById('salidaConsola');

// Función auxiliar para registrar logs visuales
function log(mensaje) {
  salidaConsola.textContent += mensaje + '\n';
}

function limpiarConsola() {
  salidaConsola.textContent = '';
}
```

QUÉ HACE EL CÓDIGO

- `log()` añade una línea al panel y `limpiarConsola()` lo vacía antes de cada caso.
- Este ejemplo solo escribe en el panel de la página.



Paso 3 · El objeto de prueba

app.js · El objeto de prueba

```
// Objeto base de prueba (con datos primitivos y un objeto anidado)
function crearObjetoOriginal() {
  return {
    nombre: 'Ana López',
    rol: 'Desarrolladora',
    edad: 28,
    direccion: {
      ciudad: 'Santander',
      codigoPostal: '39001'
    }
  };
}
```

QUÉ HACE EL CÓDIGO

- La función devuelve un objeto **nuevo** cada vez: así cada caso parte de un original limpio.
- Tiene datos simples (`nombre` , `rol` , `edad`) y un objeto anidado (`direccion`).

Ejemplo 09 · Referencias y copias (continuación)

PARTE 3 DE 5



Paso 4 · Caso 1: asignar por referencia (extracto)

app.js · Caso 1: asignar por referencia

```
btnReferencia.addEventListener('click', () => {
  const original = crearObjetoOriginal();
  const copiaFalsa = original; // ⚠️ Solo copia la dirección en memoria (puntero)

  // Modificamos a través de la variable copiaFalsa
  copiaFalsa.nombre = 'Carlos Gómez';
  copiaFalsa.direccion.ciudad = 'Torrelavega';

  // (aquí se pinta el resultado en la tarjeta con innerHTML)
});
```

QUÉ HACE EL CÓDIGO

- `const copiaFalsa = original` copia solo la referencia, no el objeto.
- Al cambiar `copiaFalsa.nombre`, `original.nombre` también cambia: son el mismo objeto.
- `original === copiaFalsa` es `true` por esa razón.



Paso 5 · Caso 2: copia superficial con spread (extracto)

app.js · Caso 2: copia superficial con spread

```
btnSpread.addEventListener('click', () => {
  const original = crearObjetoOriginal();
  const copiaSuperficial = { ...original }; // Duplica el primer nivel

  // Modificamos una propiedad del primer nivel
  copiaSuperficial.nombre = 'Beatriz Sáez';

  // ⚠️ PELIGRO: Modificamos el objeto anidado
  copiaSuperficial.direccion.ciudad = 'Laredo';

  // (aquí se pinta el resultado en la tarjeta con innerHTML)
});
```

QUÉ HACE EL CÓDIGO

- `{ ...original }` crea un objeto nuevo con las propiedades del primer nivel.
- Cambiar `copiaSuperficial.nombre` no afecta al original.
- Pero `direccion` sigue siendo el mismo objeto en los dos: cambiar `copiaSuperficial.direccion.ciudad` cambia también el original.

Ejemplo 09 · Referencias y copias (continuación)

PARTE 4 DE 5



Paso 6 · Caso 3: copia profunda (extracto)

app.js · Caso 3: copia profunda

```
btnClone.addEventListener('click', () => {
  const original = crearObjetoOriginal();
  const copiaProfunda = structuredClone(original); // Duplica todos los niveles recursivamente

  // Modificamos tanto el primer nivel como el nivel anidado
  copiaProfunda.nombre = 'Elena Ruiz';
  copiaProfunda.direccion.ciudad = 'Castro Urdiales';

  // (aquí se pinta el resultado en la tarjeta con innerHTML)
});
```

QUÉ HACE EL CÓDIGO

- `structuredClone(original)` duplica el objeto y todos sus niveles.
- Cambiar la copia, en cualquier nivel, no toca el original.
- Limitación: no puede clonar funciones; con métodos lanza un error. [4]



Paso 7 · Guardar, abrir y comprobar

7

Ejecutar el ejemplo

- Guarda `app.js` con `Ctrl + S`.
- Comprueba en Laragon que Apache está encendido.
- Abre `http://localhost/js/objetos/ejemplos/09_referencias_y_copias/index.html` y pulsa los botones.
- Abre la consola con `F12` y compara con el panel.

Ejemplo 09 · Referencias y copias (continuación)

PARTE 5 DE 5



Comprueba y practica

RESULTADO ESPERADO



QUÉ DEBES VER

- Cada botón escribe en el panel el resultado de su caso.
- Solo la copia profunda deja intacto el nivel anidado.

PRUEBA TÚ

- Cambia `copiaFalsa.edad` y comprueba el valor de `original.edad`.
- Añade un array `notas` al objeto y comprueba qué ocurre al hacer `push` con cada tipo de copia.
- Comprueba el resultado de `copiaProfunda === original`.



Ejemplo 10 · Desestructuración y spread

TEMA 10 · EXTRAER Y COMBINAR PROPIEDADES

OBJETIVO TEÓRICO

- La **desestructuración** extrae propiedades de un objeto en variables sueltas: `const { nombre, edad } = usuario` .
- Se pueden usar **alias** (`nombre: nombreCompleto`), **valores por defecto** (`pais = "España"`) y extraer datos de objetos **anidados**.
- El operador **rest** (`...resto`) recoge las propiedades que sobran; el **spread** (`{ ...a, ...b }`) combina objetos.

OBJETIVO PRÁCTICO · LO QUE TIENES QUE HACER

- Extraer propiedades con la desestructuración básica.
- Usar alias y valores por defecto.
- Extraer datos de niveles anidados.
- Separar propiedades con rest y fusionar objetos con spread.
- Usar desestructuración en los parámetros de una función.



Paso 1 · Preparar la carpeta

1 Crear la carpeta y los archivos

- Crea la carpeta `10_desestructuracion_spread` dentro de `C:\laragon\www\js\objetos\ejemplos\` .
- Copia en ella `index.html` y `style.css` , que ya están realizados.
- Crea el archivo `app.js` vacío y ábrelo con Visual Studio Code.
- En cada paso siguiente, escribe en `app.js` el código que se muestra.

Ejemplo 10 · Desestructuración y spread (continuación)

PARTE 2 DE 6



Paso 2 · Utilidades del panel

app.js · Utilidades del panel

```
// Elementos del DOM
const btnBasica = document.getElementById('btnBasica');
const btnAliasDefault = document.getElementById('btnAliasDefault');
const btnAnidada = document.getElementById('btnAnidada');
const btnRestSpread = document.getElementById('btnRestSpread');
const btnParametros = document.getElementById('btnParametros');
const cajaResultado = document.getElementById('cajaResultado');
const salidaConsola = document.getElementById('salidaConsola');

function log(msg) {
  salidaConsola.textContent += msg + '\n';
}

function limpiarConsola() {
  salidaConsola.textContent = '';
}
```

QUÉ HACE EL CÓDIGO

- `log()` añade una línea al panel y `limpiarConsola()` lo vacía antes de cada demostración.



Paso 3 · El objeto usuario

app.js · El objeto usuario

```
// Objeto base de usuario
const usuario = {
  id: 101,
  nombre: 'Lucía Fernández',
  edad: 26,
  rol: 'editor',
  ubicacion: {
    ciudad: 'Santander',
    comunidad: 'Cantabria',
    coordenadas: { lat: 43.4623, lng: -3.8099 }
  },
  contacto: {
    email: 'lucia@ejemplo.com',
    telefono: '+34 600 000 000'
  },
  tokenSecreto: 'xyz987tokenPrivado'
};
```

QUÉ HACE EL CÓDIGO

- `usuario` tiene datos simples, dos objetos anidados (`ubicacion` y `contacto`) y un dato sensible, `tokenSecreto`.

Ejemplo 10 · Desestructuración y spread (continuación)

PARTE 3 DE 6



Paso 4 · 1. Extracción básica (extracto)

app.js · 1. Extracción básica

```
btnBasica.addEventListener('click', () => {
  // Forma tradicional:
  // const nombre = usuario.nombre;
  // const edad = usuario.edad;
  // const rol = usuario.rol;

  // Con desestructuración ES6+:
  const { nombre, edad, rol } = usuario;

  // (aquí se pinta el resultado en la tarjeta con innerHTML)
});
```

QUÉ HACE EL CÓDIGO

- `const { nombre, edad, rol } = usuario` crea tres variables con los valores de las propiedades del mismo nombre.
- Equivale a escribir tres líneas del tipo `const nombre = usuario.nombre;`.



Paso 5 · 2. Alias y valores por defecto (extracto)

app.js · 2. Alias y valores por defecto

```
btnAliasDefault.addEventListener('click', () => {
  // Renombramos 'nombre' como 'nombreCompleto'
  // 'pais' no existe en 'usuario', por lo que toma el valor por defecto 'España'
  // 'rol' existe, por lo que NO usa el valor por defecto
  const {
    nombre: nombreCompleto,
    pais = 'España',
    rol = 'invitado',
    apellidos = 'No especificado'
  } = usuario;

  // (aquí se pinta el resultado en la tarjeta con innerHTML)
});
```

QUÉ HACE EL CÓDIGO

- `nombre: nombreCompleto` extrae `nombre` y lo guarda con otro nombre.
- `pais = "España"`: si la propiedad no existe, toma el valor por defecto.
- `rol = "invitado"`: como `rol` existe, **prevalece el valor del objeto**.

Ejemplo 10 · Desestructuración y spread (continuación)

PARTE 4 DE 6



Paso 6 · 3. Desestructuración anidada (extracto)

app.js · 3. Desestructuración anidada

```
btnAnidada.addEventListener('click', () => {
  // Extraemos claves de niveles inferiores en una sola instrucción
  const {
    ubicacion: { ciudad, coordenadas: { lat, lng } },
    contacto: { email }
  } = usuario;

  // (aquí se pinta el resultado en la tarjeta con innerHTML)
});
```

QUÉ HACE EL CÓDIGO

- Se puede entrar en los niveles internos dentro del mismo patrón: `ubicacion: { ciudad, coordenadas: { lat, lng } }`.
- Solo se crean variables con los nombres finales (`ciudad`, `lat`, `lng`, `email`). `ubicacion` y `coordenadas` no quedan definidas.



Paso 7 · 4. Rest y fusión con spread (extracto)

app.js · 4. Rest y fusión con spread

```
btnRestSpread.addEventListener('click', () => {
  // 1. Rest: separar datos sensibles del resto
  const { tokenSecreto, id, ...perfilPublico } = usuario;

  // 2. Spread: Fusionar configuración por defecto con personalización
  const configDefecto = { tema: 'claro', notificaciones: true, idioma: 'es', fuente: 'sans-serif' };
  const configPersonalizada = { tema: 'oscuro', idioma: 'en' };

  // Las propiedades de la derecha sobreescriben a las de la izquierda
  const configFinal = { ...configDefecto, ...configPersonalizada, fechaActualizacion: '2026-11-13' };

  // (aquí se pinta el resultado en la tarjeta con innerHTML)
});
```

QUÉ HACE EL CÓDIGO

- `const { tokenSecreto, id, ...perfilPublico } = usuario` separa dos propiedades y guarda **el resto** en `perfilPublico`.
- `{ ...configDefecto, ...configPersonalizada }` copia las propiedades de los dos objetos en uno nuevo.
- Si una clave se repite, **gana la de la derecha**: `tema` queda «oscuro» e `idioma` «en».

Ejemplo 10 · Desestructuración y spread (continuación)

PARTE 5 DE 6



Paso 8 · 5. Parámetros desestructurados (extracto)

app.js · 5. Parámetros desestructurados

```
btnParametros.addEventListener('click', () => {
  // En lugar de function renderTarjeta(usuario) y usar usuario.nombre...
  function renderTarjeta({ nombre, edad = 18, ubicacion: { ciudad = 'Desconocida' } = {} }) {
    return `
      <div style="border: 2px dashed #38bdf8; padding: 1rem; border-radius: 8px;">
        <h4 style="margin: 0; color: #38bdf8;">👤 ${nombre}</h4>
        <p style="margin: 0.25rem 0;">🎂 Edad: ${edad} años</p>
        <p style="margin: 0;">📍 Ciudad: ${ciudad}</p>
      </div>
    `;
  }

  const html1 = renderTarjeta(usuario);

  const html2 = renderTarjeta({ nombre: 'Marta Vega' });

  // (aquí se pinta el resultado en la tarjeta con innerHTML)
});
```

QUÉ HACE EL CÓDIGO

- `function renderTarjeta({ nombre, edad = 18, ... })` desestructura el objeto que recibe, con valores por defecto.
- Con el objeto completo usa sus datos. Con `{ nombre: "Marta Vega" }` usa los valores por defecto para lo que falta.
- El `= {}` tras `ubicacion: { ... }` evita un error si el objeto no trae `ubicacion`.



Paso 9 · Guardar, abrir y comprobar

9

Ejecutar el ejemplo

- Guarda `app.js` con `Ctrl + S`.
- Comprueba en Laragon que Apache está encendido.
- Abre `http://localhost/js/objetos/ejemplos/10_desestructuracion_spread/index.html` y pulsa los botones.
- Abre la consola con `F12` y compara con el panel.

Ejemplo 10 · Desestructuración y spread (continuación)

PARTE 6 DE 6



Comprueba y practica

RESULTADO ESPERADO



QUÉ DEBES VER

- Cada botón escribe en el panel qué variables ha creado y con qué valores.
- En el botón 4, `perfilPublico` no contiene `id` ni `tokenSecreto`.

PRUEBA TÚ

- Extrae `rol` con el alias `puesto`.
- Añade a `configPersonalizada` la clave `fuente` y comprueba qué valor gana.
- Llama a `renderTarjeta({})` y observa el resultado.



Ejemplo 11 · Getters, setters y privacidad

TEMA 11 · PROPIEDADES CALCULADAS Y DATOS PROTEGIDOS

OBJETIVO TEÓRICO

- Un **getter** (`get`) se lee como una propiedad, pero ejecuta una función: sirve para valores calculados.
- Un **setter** (`set`) se asigna como una propiedad, pero ejecuta una función: sirve para **validar** antes de guardar.
- Un objeto literal tiene todos sus miembros públicos. Para proteger datos se usa una **clausura**: una función factoría cuyas variables internas no se pueden alcanzar desde fuera. `Object.freeze` impide modificar un objeto.

OBJETIVO PRÁCTICO · LO QUE TIENES QUE HACER

- Crear un objeto con getters y un setter con validación.
- Provocar errores de validación y capturarlos con `try...catch`.
- Crear una función factoría con una variable privada.
- Congelar un objeto de configuración.



Paso 1 · Preparar la carpeta

1 Crear la carpeta y los archivos

- Crea la carpeta `11_getters_setters_privacidad` dentro de `C:\laragon\www\js\objetos\ejemplos\`.
- Copia en ella `index.html` y `style.css`, que ya están realizados.
- Crea el archivo `app.js` vacío y ábrelo con Visual Studio Code.
- En cada paso siguiente, escribe en `app.js` el código que se muestra.

Ejemplo 11 · Getters, setters y privacidad (continuación)

PARTE 2 DE 8



Paso 2 · Utilidades del panel

app.js · Utilidades del panel

```
// Elementos del DOM
const btnGetters = document.getElementById('btnGetters');
const btnSettersValidos = document.getElementById('btnSettersValidos');
const btnSetterInvalido = document.getElementById('btnSetterInvalido');
const btnClausuraPrivada = document.getElementById('btnClausuraPrivada');
const btnObjectFreeze = document.getElementById('btnObjectFreeze');
const cajaResultado = document.getElementById('cajaResultado');
const salidaConsola = document.getElementById('salidaConsola');

function log(msg) {
  salidaConsola.textContent += msg + '\n';
}

function limpiarConsola() {
  salidaConsola.textContent = '';
}
```

QUÉ HACE EL CÓDIGO

- `log()` añade una línea al panel y `limpiarConsola()` lo vacía antes de cada demostración.

Ejemplo 11 · Getters, setters y privacidad (continuación)

PARTE 3 DE 8



Paso 3 · El objeto producto con getters y setter

app.js · El objeto producto con getters y setter

```
const producto = {
  _nombre: 'Teclado Mecánico RGB',
  _precioBase: 80,
  _iva: 0.21,

  // Getter: Se lee como una propiedad normal (sin paréntesis producto.precioFinal)
  get precioFinal() {
    return Number((this._precioBase * (1 + this._iva)).toFixed(2));
  },

  get etiqueta() {
    return `${this._nombre.toUpperCase()} - ${this.precioFinal}€ (IVA incl.)`;
  },

  // Setter: Valida antes de guardar
  set precioBase(nuevoPrecio) {
    if (typeof nuevoPrecio !== 'number' || isNaN(nuevoPrecio)) {
      throw new TypeError('El precio base debe ser un valor numérico.');
    }
    if (nuevoPrecio <= 0) {
      throw new RangeError('El precio base debe ser estrictamente mayor a 0€.');
    }
    this._precioBase = nuevoPrecio;
  },

  get precioBase() {
    return this._precioBase;
  }
};
```

QUÉ HACE EL CÓDIGO

- `_nombre`, `_precioBase` y `_iva` guardan los datos. El guion bajo es solo una convención: avisa de que no deben tocarse desde fuera.
- `get precioFinal()` calcula el precio con IVA cada vez que se lee. Se usa sin paréntesis.
- `get etiqueta()` reutiliza otro getter, `precioFinal`.
- `set precioBase(...)` comprueba que sea un número mayor que 0 y lanza `TypeError` o `RangeError` si no lo es.
- `get precioBase()` permite leer lo que guarda el setter.

Ejemplo 11 · Getters, setters y privacidad (continuación)

PARTE 4 DE 8



Paso 4 · 1. Getters computados (extracto)

app.js · 1. Getters computados

```
btnGetters.addEventListener('click', () => {
  // (aquí se pinta el resultado en la tarjeta con innerHTML)
});
```

QUÉ HACE EL CÓDIGO

- `producto.precioFinal` y `producto.etiqueta` se leen sin paréntesis, como propiedades, pero ejecutan código.



Paso 5 · 2. Setter con un valor válido (extracto)

app.js · 2. Setter con un valor válido

```
btnSettersValidos.addEventListener('click', () => {
  producto.precioBase = 120; // Pasa por el setter

  // (aquí se pinta el resultado en la tarjeta con innerHTML)
});
```

QUÉ HACE EL CÓDIGO

- `producto.precioBase = 120` pasa por el setter, que valida y guarda.
- El precio final se recalcula solo, porque es un getter.

Ejemplo 11 · Getters, setters y privacidad (continuación)

PARTE 5 DE 8



Paso 6 · 3. Setter con valores inválidos (extracto)

app.js · 3. Setter con valores inválidos

```
btnSetterInvalido.addEventListener('click', () => {
  try {
    producto.precioBase = -45;
  } catch (error) {
    // (se escribe el resultado en el panel)
  }

  try {
    producto.precioBase = 'gratis';
  } catch (error) {
    // (se escribe el resultado en el panel)
  }

  // (aquí se pinta el resultado en la tarjeta con innerHTML)
});
```

QUÉ HACE EL CÓDIGO

- Se intenta asignar `-45` y `"gratis"`: el setter lanza un error en cada caso.
- `try...catch` captura el error y muestra `error.name` y `error.message`.
- El valor del objeto no cambia: el estado sigue siendo seguro.

Ejemplo 11 · Getters, setters y privacidad (continuación)

PARTE 6 DE 8



Paso 7 · 4. Privacidad real con una clausura (extracto)

app.js · 4. Privacidad real con una clausura

```
btnClausuraPrivada.addEventListener('click', () => {
  // Función factoría: crea un objeto pero las variables let quedan atrapadas en el scope léxico
  function crearCuenta(titular, saldoInicial) {
    let saldoPrivado = saldoInicial; // 🔒 Variable inaccesible directamente desde fuera

    return {
      titular: titular,
      ingresar(cantidad) {
        if (cantidad <= 0) return 'Cantidad inválida';
        saldoPrivado += cantidad;
        return `Ingresados ${cantidad}€. Nuevo saldo: ${saldoPrivado}€`;
      },
      retirar(cantidad) {
        if (cantidad > saldoPrivado) return 'Fondos insuficientes';
        saldoPrivado -= cantidad;
        return `Retirados ${cantidad}€. Saldo restante: ${saldoPrivado}€`;
      },
      consultarSaldo() {
        return saldoPrivado;
      }
    };
  }

  const miCuenta = crearCuenta('Ramón Abramo', 1000);
  miCuenta.saldoPrivado = 9999999; // Solo añade una propiedad irrelevante

  // (aquí se pinta el resultado en la tarjeta con innerHTML)
});
```

QUÉ HACE EL CÓDIGO

- `crearCuenta` es una función factoría: devuelve un objeto literal.
- `saldoPrivado` es una variable de la función, no una propiedad del objeto: nadie puede leerla desde fuera.
- Los métodos devueltos sí pueden usarla, porque la recuerdan (clausura).
- `miCuenta.saldoPrivado = 9999999` solo crea una propiedad nueva sin importancia; el saldo real no cambia.

Ejemplo 11 · Getters, setters y privacidad (continuación)

PARTE 7 DE 8



Paso 8 · 5. Congelar con Object.freeze (extracto)

app.js · 5. Congelar con Object.freeze

```
btnObjectFreeze.addEventListener('click', () => {
  const configuracionApp = Object.freeze({
    servidor: 'https://api.santander.es',
    puerto: 8080,
    modo: 'produccion'
  });

  configuracionApp.version = '1.0';

  configuracionApp.puerto = 3000;

  // (aquí se pinta el resultado en la tarjeta con innerHTML)
});
```

QUÉ HACE EL CÓDIGO

- `Object.freeze` impide añadir, borrar y modificar propiedades.
- Los intentos se ignoran en silencio (en modo estricto lanzarían un `TypeError`).
- `Object.isFrozen(objeto)` devuelve `true`.



Paso 9 · Guardar, abrir y comprobar

9

Ejecutar el ejemplo

- Guarda `app.js` con `Ctrl + S`.
- Comprueba en Laragon que Apache está encendido.
- Abre `http://localhost/js/objetos/ejemplos/11_getters_setters_privacidad/index.html` y pulsa los botones.
- Abre la consola con `F12` y compara con el panel.

Ejemplo 11 · Getters, setters y privacidad (continuación)

PARTE 8 DE 8



Comprueba y practica

RESULTADO ESPERADO



QUÉ DEBES VER

- Los botones muestran la lectura con getter, la escritura válida, los errores capturados, la privacidad real y el objeto congelado.
- Tras un error en el setter, el precio base no ha cambiado.

PRUEBA TÚ

- Añade un getter `descuento10` que devuelva el precio final menos un 10 %.
- Añade al setter una comprobación: el precio no puede superar 10000.
- Añade a la cuenta un método `historial()` que devuelva una copia de los movimientos.



Ejemplo 12 · JSON y localStorage

TEMA 12 · SERIALIZAR Y PERSISTIR DATOS

OBJETIVO TEÓRICO

- **JSON** es un formato de texto para intercambiar datos. `JSON.stringify` convierte un objeto en texto y `JSON.parse` hace el camino contrario. [5]
- JSON no puede guardar funciones, `undefined` ni símbolos, y las fechas pasan a texto. [6]
- `localStorage` guarda pares clave-valor en el navegador, **siempre como texto**: por eso se usa junto con JSON. [7]

OBJETIVO PRÁCTICO · LO QUE TIENES QUE HACER

- Serializar un objeto con `JSON.stringify`, en compacto y con sangría.
- Reconstruir un objeto con `JSON.parse`.
- Comprobar qué datos se pierden al pasar por JSON.
- Guardar, leer y borrar un objeto en `localStorage`.



Paso 1 · Preparar la carpeta

1 Crear la carpeta y los archivos

- Crea la carpeta `12_json_y_localstorage` dentro de `C:\laragon\www\js\objetos\ejemplos\`.
- Copia en ella `index.html` y `style.css`, que ya están realizados.
- Crea el archivo `app.js` vacío y ábrelo con Visual Studio Code.
- En cada paso siguiente, escribe en `app.js` el código que se muestra.

Ejemplo 12 · JSON y localStorage (continuación)

PARTE 2 DE 7



Paso 2 · Utilidades y clave de almacenamiento

app.js · Utilidades y clave de almacenamiento

```
// Elementos del DOM
const btnStringify = document.getElementById('btnStringify');
const btnParse = document.getElementById('btnParse');
const btnLimites = document.getElementById('btnLimites');
const btnGuardarLocal = document.getElementById('btnGuardarLocal');
const btnCargarLocal = document.getElementById('btnCargarLocal');
const btnBorrarLocal = document.getElementById('btnBorrarLocal');
const cajaResultado = document.getElementById('cajaResultado');
const salidaConsola = document.getElementById('salidaConsola');

const CLAVE_STORAGE = 'ramon_curso_usuario_demo';

function log(msg) {
  salidaConsola.textContent += msg + '\n';
}

function limpiarConsola() {
  salidaConsola.textContent = '';
}
```

QUÉ HACE EL CÓDIGO

- `CLAVE_STORAGE` es el nombre con el que se guardará el objeto en `localStorage`.
- `log()` y `limpiarConsola()` escriben y vacían el panel de la página.

Ejemplo 12 · JSON y localStorage (continuación)

PARTE 3 DE 7



Paso 3 · 1. JSON.stringify (extracto)

app.js · 1. JSON.stringify

```
btnStringify.addEventListener('click', () => {
  const usuario = {
    id: 42,
    nombre: 'Elena Castro',
    activo: true,
    habilidades: ['HTML5', 'CSS3', 'JavaScript'],
    direccion: {
      ciudad: 'Santander',
      cp: '39005'
    }
  };

  log(usuario.toString()); // [object Object]

  // Serialización sin formateo
  const jsonCompacto = JSON.stringify(usuario);

  // Serialización formateada con 2 espacios
  const jsonFormateado = JSON.stringify(usuario, null, 2);

  // (aquí se pinta el resultado en la tarjeta con innerHTML)
});
```

QUÉ HACE EL CÓDIGO

- `usuario.toString()` da `[object Object]`: un objeto no es texto.
- `JSON.stringify(usuario)` genera el texto compacto, apto para enviar por red.
- `JSON.stringify(usuario, null, 2)` genera el mismo texto con sangría de 2 espacios, más fácil de leer.

Ejemplo 12 · JSON y localStorage (continuación)

PARTE 4 DE 7



Paso 4 · 2. JSON.parse (extracto)

app.js · 2. JSON.parse

```
btnParse.addEventListener('click', () => {
  const cadenaJson = '{"nombre":"Marcos Prieto","ciudad":"Santander","matricula":2026,"aprobado":tru
e}';

  try {
    const objetoRecuperado = JSON.parse(cadenaJson);

    // (aquí se pinta el resultado en la tarjeta con innerHTML)
  } catch (error) {
    // (se escribe el resultado en el panel)
  }
});
```

QUÉ HACE EL CÓDIGO

- `cadenaJson` es un texto con formato JSON: claves y textos entre comillas dobles.
- `JSON.parse(cadenaJson)` devuelve un objeto con el que se puede trabajar.
- Va dentro de `try...catch` porque `JSON.parse` lanza un error si el texto no es JSON válido.



Paso 5 · 3. Lo que JSON no conserva (extracto)

app.js · 3. Lo que JSON no conserva

```
btnLimites.addEventListener('click', () => {
  const objetoComplejo = {
    titulo: 'Lección de Objetos',
    fechaCreacion: new Date(),        // Pasa a string ISO, no a Date
    saludar: function() { return '¡Hola!'; }, // ❌ Se descarta por completo
    datoVacio: undefined,             // ❌ Se descarta por completo
    valorEspecial: NaN,               // Pasa a null
    simbolo: Symbol('id')             // ❌ Se descarta por completo
  };

  const json = JSON.stringify(objetoComplejo, null, 2);

  const rehidratado = JSON.parse(json);

  // (aquí se pinta el resultado en la tarjeta con innerHTML)
});
```

QUÉ HACE EL CÓDIGO

- Funciones, `undefined` y símbolos se descartan al serializar.
- `NaN` pasa a `null` y la fecha pasa a un texto.
- Tras `JSON.parse`, `saludar` ya no existe y `fechaCreacion` es un `string`: no tiene métodos como `getFullYear()`.

Ejemplo 12 · JSON y localStorage (continuación)

PARTE 5 DE 7



Paso 6 · 4. Guardar en localStorage (extracto)

app.js · 4. Guardar en localStorage

```
btnGuardarLocal.addEventListener('click', () => {
  const perfilGuardar = {
    usuario: 'alumno_santander',
    preferencias: {
      tema: 'oscuro',
      tamanoFuente: '16px',
      sonidos: false
    },
    ultimoAcceso: new Date().toLocaleTimeString('es-ES')
  };

  // LocalStorage solo almacena cadenas de texto
  // Si hiciéramos: localStorage.setItem(CLAVE, perfilGuardar), guardaría "[object Object]"
  const jsonAAlmacenar = JSON.stringify(perfilGuardar);
  localStorage.setItem(CLAVE_STORAGE, jsonAAlmacenar);

  // (aquí se pinta el resultado en la tarjeta con innerHTML)
});
```

QUÉ HACE EL CÓDIGO

- `localStorage` solo guarda texto: si se guardara el objeto directamente, se guardaría `"[object Object]"`.
- `JSON.stringify(perfilGuardar)` lo convierte antes de guardarlo con `setItem(clave, texto)`.



Paso 7 · 5. Cargar desde localStorage (extracto)

app.js · 5. Cargar desde localStorage

```
btnCargarLocal.addEventListener('click', () => {
  const textoRecuperado = localStorage.getItem(CLAVE_STORAGE);

  if (!textoRecuperado) {
    // (aquí se pinta el resultado en la tarjeta con innerHTML)
    return;
  }

  const objetoConfig = JSON.parse(textoRecuperado);

  // (aquí se pinta el resultado en la tarjeta con innerHTML)
});
```

QUÉ HACE EL CÓDIGO

- `getItem(clave)` devuelve el texto guardado, o `null` si no hay nada. Por eso se comprueba antes de usarlo.
- `JSON.parse(textoRecuperado)` reconstruye el objeto y ya se pueden leer sus propiedades, incluso las anidadas.

Ejemplo 12 · JSON y localStorage (continuación)

PARTE 6 DE 7



Paso 8 · 6. Borrar de localStorage (extracto)

app.js · 6. Borrar de localStorage

```
btnBorrarLocal.addEventListener('click', () => {
  localStorage.removeItem(CLAVE_STORAGE);

  // (aquí se pinta el resultado en la tarjeta con innerHTML)
});
```

QUÉ HACE EL CÓDIGO

- `removeItem(clave)` elimina esa clave de `localStorage`.



Paso 9 · Guardar, abrir y comprobar

9

Ejecutar el ejemplo

- Guarda `app.js` con `Ctrl + S`.
- Comprueba en Laragon que Apache está encendido.
- Abre `http://localhost/js/objetos/ejemplos/12_json_y_localstorage/index.html` y pulsa los botones.
- Abre la consola con `F12` y compara con el panel.

Ejemplo 12 · JSON y localStorage (continuación)

PARTE 7 DE 7



Comprueba y practica

RESULTADO ESPERADO



QUÉ DEBES VER

- Tras guardar y recargar la página, «Cargar» recupera los datos aunque se cierre la pestaña.
- Tras «Limpiar», «Cargar» avisa de que no hay datos guardados.

PRUEBA TÚ

- Guarda el objeto, abre `F12` → **Aplicación** → **Almacenamiento local** y localiza la clave.
- Añade una propiedad nueva al perfil y comprueba que se guarda.
- Guarda un array de objetos en lugar de un objeto.



Resumen de conceptos

REPASO DE LOS 12 EJEMPLOS

Cada ejemplo trabaja un concepto. Este cuadro los reúne para que puedas repasar.

Ejemplo	Concepto	Qué tienes que recordar
01	Creación y anatomía	Propiedades y métodos entre llaves; tres formas de crear un objeto
02	Valores y claves	Cualquier tipo de valor; comillas si hay espacios o guiones; claves calculadas `[expr]`
03	Lectura	`obj.clave`, `obj[variable]` y `obj?.a?.b`
04	Modificar	Asignar, añadir, `delete`, `in` y `Object.freeze`
05	Métodos y `this`	`this` es el objeto que llama; no uses flechas como métodos
06	Anidados	Encadena puntos y corchetes; `?.` evita el error
07	Arrays de objetos	`filter`, `map`, `find` y `reduce`
08	Recorrer	`Object.keys`, `values`, `entries` y `for...in`
09	Referencias	`b = a` no copia; `{ ...a }` copia el primer nivel; `structuredClone` copia todo
10	Desestructuración	`{ a, b }`, alias, valores por defecto, rest y spread
11	Getters y setters	`get` calcula, `set` valida; la clausura da privacidad real
12	JSON y almacenamiento	`stringify`, `parse`, lo que JSON pierde y `localStorage`



Fuentes y Documentación Oficial

REFERENCIAS DE LA GUÍA · CONSULTA: 13/11/2026

Cada número entre corchetes remite a la documentación correspondiente. Los ejemplos se han ejecutado y comprobado en el navegador.



Fuentes y Documentación Oficial

[1] Trabajando con objetos · MDN Web Docs · developer.mozilla.org/es/docs/Web/JavaScript/Guide/Trabajando_con_objectos

[2] this · MDN Web Docs · developer.mozilla.org/es/docs/Web/JavaScript/Referencia/Operadores/this

[3] Funciones flecha · MDN Web Docs · developer.mozilla.org/es/docs/Web/JavaScript/Reference/Functions/Arrow_functions

[4] Copia profunda en JavaScript con structuredClone · web.dev · web.dev/articles/structured-clone

[5] JSON · MDN Web Docs · developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/JSON

[6] JSON.stringify() · MDN Web Docs · developer.mozilla.org/JavaScript/Reference/Global_Objects/JSON/stringify

[7] Window.localStorage · MDN Web Docs · developer.mozilla.org/es/docs/Web/API/Window/localStorage